

Polynomials

Program Design Discussion

This program is designed using a modular, procedural approach with linked data structures to separate responsibilities and improve readability, maintainability, and correctness. The program allows the user to input between 0 and 10 polynomials, then computes and displays both their sum and product. The main control flow is handled by the `runProgram` function, which manages user interaction, input validation, and repeated execution until termination. Each polynomial is represented as a linked list of terms, and all operations such as insertion, addition, multiplication, parsing, and display are handled through well-defined helper functions. The main function remains minimal and simply calls `runProgram` to begin execution. Input validation is handled carefully to ensure robustness. The number of polynomials must be within a defined range. Each polynomial must be entered in a valid algebraic format, supporting integers, variables, exponents, and signs. Invalid inputs are rejected and the user is repeatedly prompted until a correct format is provided. Parsing logic ensures that terms are interpreted correctly, including implicit coefficients and exponents, while also handling whitespace and formatting inconsistencies.

Data Structure Design Discussion

Polynomial Data Structure

Each polynomial is implemented as a singly linked list using the `ATerm` structure. Each node contains:

- `int` coefficient: the numerical multiplier of the term
- `int` exponent: the power of the variable
- `ATerm*` next: pointer to the next term in the list

The polynomial is maintained in descending order of exponents. This ordering simplifies operations such as addition and display while ensuring consistent structure. The use of dynamic memory allows polynomials to grow as needed without fixed size limitations.

Term Insertion and Management

The `insertTerm` function is responsible for maintaining the integrity of the polynomial. It ensures that terms are inserted in the correct order and combines like terms when exponents match. If combining terms results in a coefficient of zero, the node is removed to maintain a clean representation. Additional helper functions such as `createNode`, `clearPolynomial`, and `copyPolynomial` manage memory allocation, deallocation, and duplication of polynomials, ensuring no memory leaks occur during execution.

Polynomial Operation Structure

The program supports two primary operations: addition and multiplication.

- Addition is performed by traversing two polynomials simultaneously, comparing exponents, and inserting resulting terms into a new polynomial.
- Multiplication is performed using a nested traversal, where each term of one polynomial is multiplied with every term of the other, and results are accumulated using `insertTerm`.

Both operations return new dynamically allocated polynomials, preserving the original inputs and ensuring functional separation.

Parsing and Input Handling Structure

Polynomial input is processed using a series of parsing functions. The `parsePolynomial` function reads an entire string and extracts individual terms using `parseTerm`. Supporting functions such as `skipSpaces` and `readUnsignedInteger` assist in handling formatting and numeric extraction. This layered parsing approach allows the program to handle flexible user input while maintaining strict validation rules.

Display and Output Structure

The program includes dedicated display functions to format output clearly. `displayTerm` handles individual term formatting, including sign and exponent rules. `displayPolynomial` outputs a full polynomial, while `displayBracketPolynomial` adds visual grouping for expressions. Higher-level functions `displaySum` and `displayProduct` show both the full expression and the final computed result, improving readability for the user.

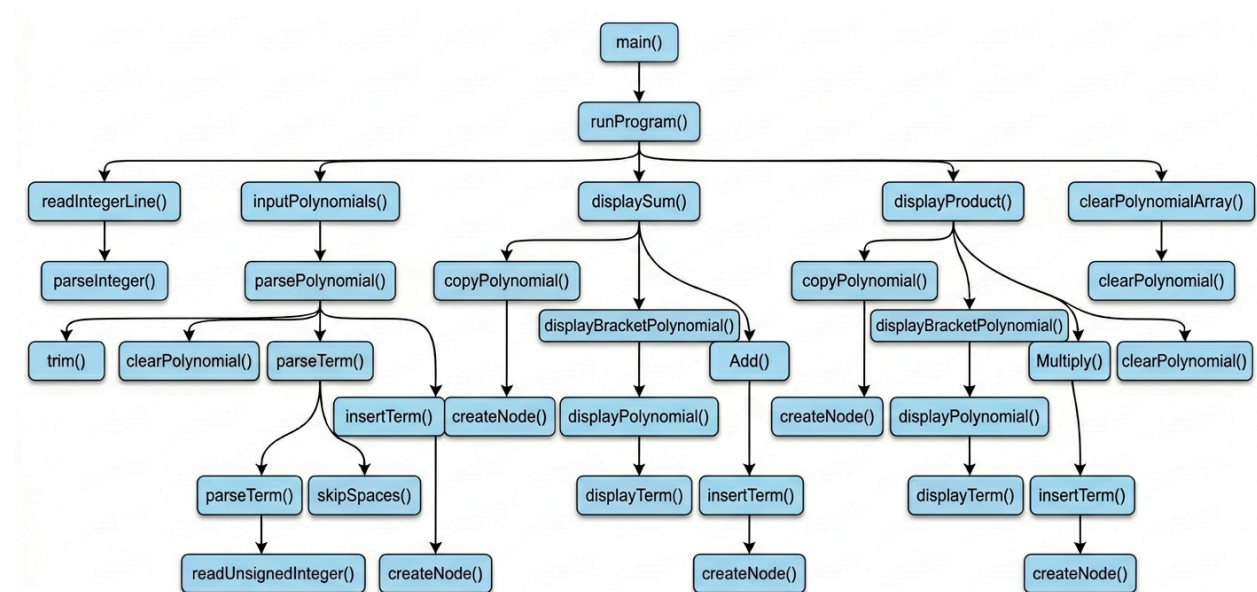
Program Control Structure

The overall program flow is controlled by the `runProgram` function, which contains:

- `int polynomialCount`: number of polynomials entered by the user
- `Polynomial polynomials[MAX_POLYNOMIALS]`: array storing all input polynomials

The function repeatedly prompts the user, processes input, computes results, and clears memory before restarting. This centralized control structure ensures that all components of the program work together cohesively while maintaining separation between input, processing, and output logic.

Hierarchy Chart:

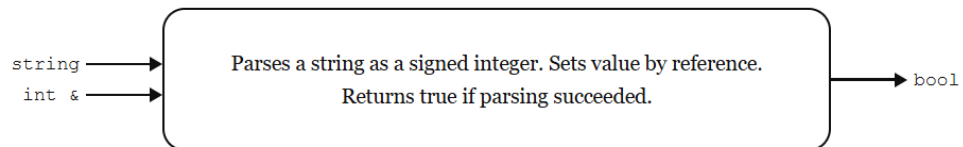


Procedural Specification:

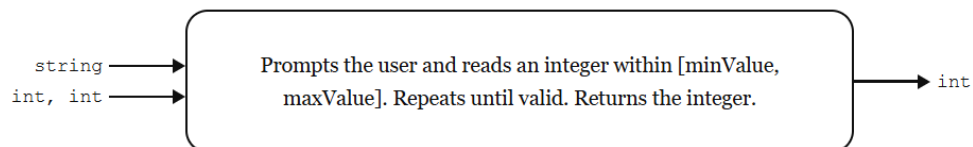
Function trim



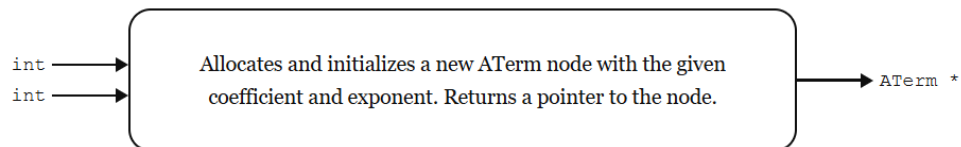
Function parseInt



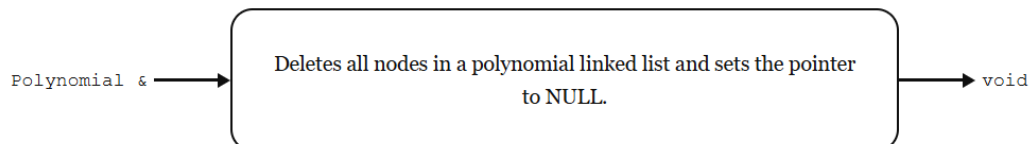
Function readIntegerLine



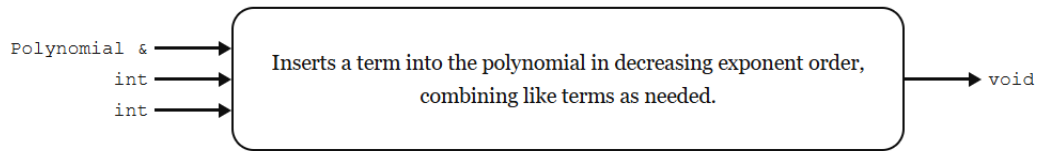
Function createNode



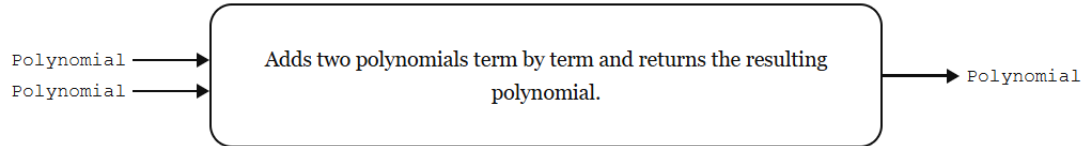
Function clearPolynomial



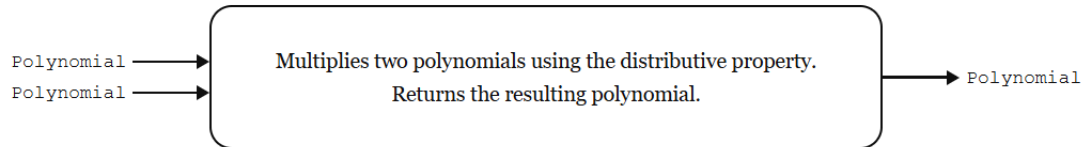
Function insertTerm



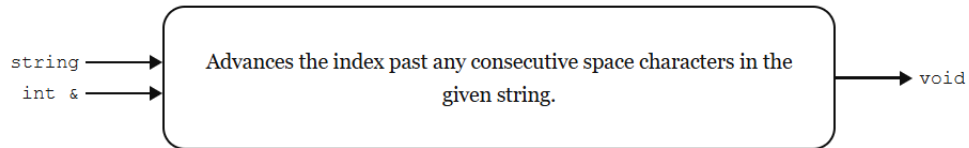
Function Add



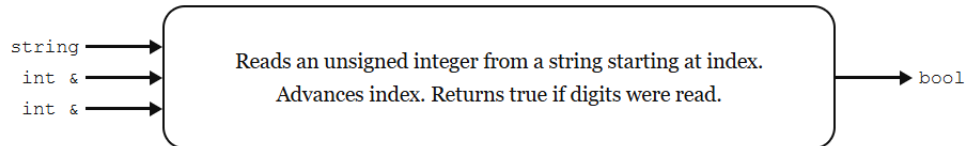
Function Multiply



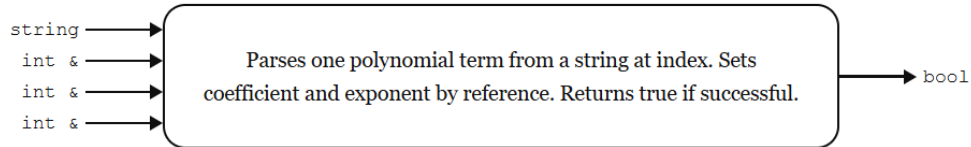
Function skipSpaces



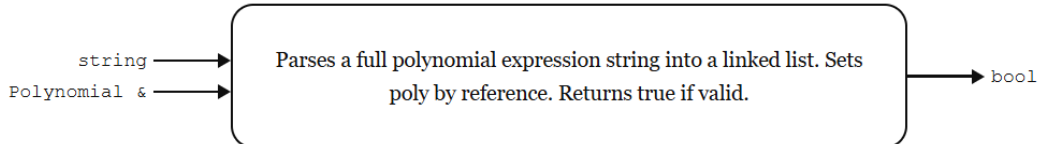
Function readUnsignedInteger



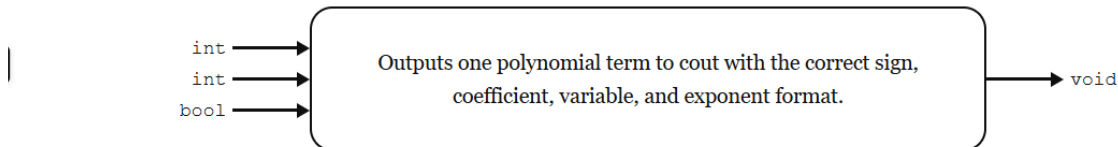
Function parseTerm



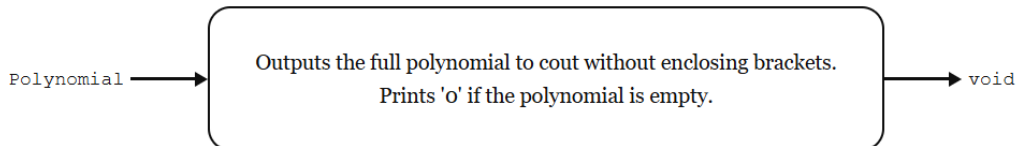
Function parsePolynomial



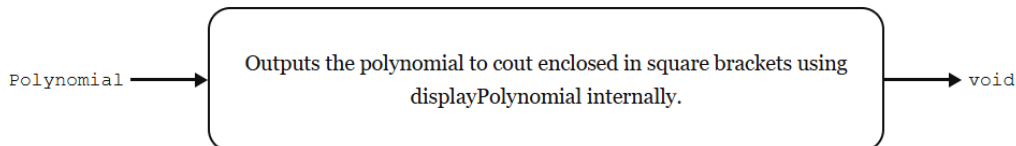
Function displayTerm



Function displayPolynomial



Function displayBracketPolynomial



Test Cases with Expected Outputs

Test Case 1 — Exit Immediately

Inputs: number of polynomials = 0

Justification: Confirms the program accepts the minimum valid polynomial count and terminates without requesting any polynomial input.

Expected Output: Program exits cleanly without displaying a sum or product.

Test Case 2 — Valid Single Constant Polynomial

Inputs: number of polynomials = 1; polynomial 1 = 5

Justification: Confirms the program correctly parses a constant term and handles the simplest nonzero polynomial input.

Expected Output: Program accepts the input, displays Sum = [5] = 5, and Product = [5] = 5.

Test Case 3 — Valid Single Variable Polynomial

Inputs: number of polynomials = 1; polynomial 1 = x

Justification: Confirms the program correctly parses a term with an implied coefficient of 1 and implied exponent of 1.

Expected Output: Program accepts the input, displays Sum = [x] = x, and Product = [x] = x.

Test Case 4 — Valid Polynomial with Multiple Ordered Terms

Inputs: number of polynomials = 1; polynomial 1 = $3x^4 + 2x^2 - 7x + 5$

Justification: Confirms the program correctly parses a standard multi-term polynomial written in descending exponent order.

Expected Output: Program accepts the input and displays the polynomial in the same simplified format.

Test Case 5 — Valid Polynomial with Terms Out of Order

Inputs: number of polynomials = 1; polynomial 1 = $5 + 2x^2 + 3x^5 - x$

Justification: Confirms terms are inserted into the linked list in descending exponent order even if entered out of order.

Expected Output: Program accepts the input and displays it reordered as $3x^5 + 2x^2 - x + 5$.

Test Case 6 — Valid Polynomial with Like Terms Combined

Inputs: number of polynomials = 1; polynomial 1 = $3x^2 + 4x^2 - x + 6$

Justification: Confirms the program combines like terms during insertion.

Expected Output: Program accepts the input and displays $7x^2 - x + 6$.

Test Case 7 — Valid Polynomial that Simplifies to Zero

Inputs: number of polynomials = 1; polynomial 1 = $x^2 - x^2$

Justification: Confirms the program correctly handles cancellation of all terms and stores the

zero polynomial.

Expected Output: Program accepts the input and displays 0 for both the sum and product.

Test Case 8 — Valid Polynomial with Extra Spaces

Inputs: number of polynomials = 1; polynomial 1 = $3x^2 - 4x + 7$

Justification: Confirms the parser correctly ignores leading, trailing, and internal extra spaces.

Expected Output: Program accepts the input and displays $3x^2 - 4x + 7$.

Test Case 9 — Valid Polynomial Using Uppercase Variable

Inputs: number of polynomials = 1; polynomial 1 = $2X^3 - X + 1$

Justification: Confirms the parser accepts both lowercase x and uppercase X.

Expected Output: Program accepts the input and displays $2x^3 - x + 1$.

Test Case 10 — Valid Polynomial with Negative Exponent

Inputs: number of polynomials = 1; polynomial 1 = $3x^{-2} + 5$

Justification: Confirms the parser accepts negative exponents.

Expected Output: Program accepts the input and displays $3x^{-2} + 5$.

Test Case 11 — Valid Maximum Polynomial Count

Inputs: number of polynomials = 10; polynomials = x, x, x, x, x, x, x, x, x, x

Justification: Confirms the program accepts the maximum allowed number of polynomials.

Expected Output: Program accepts all ten polynomials, displays the sum as $10x$, and displays the product as x^{10} .

Test Case 12 — Invalid Polynomial Count Below Range

Inputs: number of polynomials = -1, then 1

Justification: Ensures the program rejects a value below the allowed minimum.

Expected Output: Program prints an error message and reprompts until a valid count is entered.

Test Case 13 — Invalid Polynomial Count Above Range

Inputs: number of polynomials = 11, then 2

Justification: Ensures the program rejects a value above the allowed maximum.

Expected Output: Program prints an error message and reprompts until a valid count is entered.

Test Case 14 — Invalid Polynomial Count with Noninteger Input

Inputs: number of polynomials = abc, then 2

Justification: Confirms the integer input routine rejects nonnumeric input.

Expected Output: Program prints an error message and reprompts until a valid integer is entered.

Test Case 15 — Invalid Empty Polynomial Input

Inputs: number of polynomials = 1; polynomial 1 = empty line, then $x+1$

Justification: Confirms the parser rejects blank polynomial input.

Expected Output: Program prints an error message and reprompts until a valid polynomial is entered.

Test Case 16 — Invalid Polynomial with Missing Exponent Value

Inputs: number of polynomials = 1; polynomial 1 = $3x^$

Justification: Confirms the parser rejects terms with a caret but no exponent.

Expected Output: Program prints an error message and reprompts.

Test Case 17 — Invalid Polynomial with Unsupported Character

Inputs: number of polynomials = 1; polynomial 1 = $3x^2 + 4y$

Justification: Confirms the parser rejects variables other than x or X.

Expected Output: Program prints an error message and reprompts.

Test Case 18 — Invalid Polynomial with Missing Operator Between Terms

Inputs: number of polynomials = 1; polynomial 1 = $3x^2 4x + 1$

Justification: Confirms the parser requires explicit + or - between terms.

Expected Output: Program prints an error message and reprompts.

Test Case 19 — Addition of Two Polynomials with No Like Terms

Inputs: number of polynomials = 2; polynomial 1 = $3x^2 + 1$; polynomial 2 = $4x - 5$

Justification: Confirms addition works correctly when no exponents match.

Expected Output: Sum displays as $3x^2 + 4x - 4$.

Test Case 20 — Addition of Two Polynomials with Like Terms

Inputs: number of polynomials = 2; polynomial 1 = $3x^2 + 2x + 1$; polynomial 2 = $5x^2 - 2x + 4$

Justification: Confirms addition combines matching exponents correctly and removes matched linear terms that cancel.

Expected Output: Sum displays as $8x^2 + 5$.

Test Case 21 — Addition Resulting in Zero Polynomial

Inputs: number of polynomials = 2; polynomial 1 = $2x^3 - 7$; polynomial 2 = $-2x^3 + 7$

Justification: Confirms addition can produce the zero polynomial.

Expected Output: Sum displays as 0.

Test Case 22 — Multiplication of Two Simple Polynomials

Inputs: number of polynomials = 2; polynomial 1 = $x + 1$; polynomial 2 = $x + 1$

Justification: Confirms multiplication produces the expected expanded result and combines like terms properly.

Expected Output: Product displays as $x^2 + 2x + 1$.

Test Case 23 — Multiplication by Zero Polynomial

Inputs: number of polynomials = 2; polynomial 1 = $x^2 + 3x + 2$; polynomial 2 = 0

Justification: Confirms multiplying by the zero polynomial produces zero.

Expected Output: Product displays as 0.

Test Case 24 — Multiple Polynomial Sum and Product

Inputs: number of polynomials = 3; polynomial 1 = x ; polynomial 2 = $x + 1$; polynomial 3 = $x - 1$

Justification: Confirms the program correctly handles more than two polynomials for both cumulative addition and cumulative multiplication.

Expected Output: Sum displays as $3x$, and Product displays as $x^3 - x$.

Test Case 25 — Repeated Program Execution

Inputs: first run: number of polynomials = 1; polynomial 1 = $x + 1$; second run: number of polynomials = 1; polynomial 1 = $x - 1$; final input: 0

Justification: Confirms the program properly clears memory and returns to the main prompt for another complete run.

Expected Output: Program completes the first run, displays a separator line, accepts a second run correctly, and then exits cleanly after 0.

Results of Test Cases:

```
Number of polynomials -> 0  
PS C:\Users\Brent\OneDrive\Desktop\Data Structures & Algorithms\Polynomials>
```

```
Number of polynomials -> 1  
Input polynomial 1 -> 5  
Sum = [5]  
      = 5  
Product = [5]  
          = 5
```

```
Number of polynomials -> 1  
Input polynomial 1 -> x  
Sum = [x]  
      = x  
Product = [x]  
          = x
```

```
-----  
Number of polynomials ->
```

```
Number of polynomials -> 1  
Input polynomial 1 -> 5  
Sum = [5]  
      = 5  
Product = [5]  
          = 5
```

```
-----  
Number of polynomials -> 1  
Input polynomial 1 -> x  
Sum = [x]  
      = x  
Product = [x]  
          = x
```

Number of polynomials -> 1
Input polynomial 1 -> $3x^4 + 2x^2 - 7x + 5$
Sum = $[3x^4 + 2x^2 - 7x + 5]$
 = $3x^4 + 2x^2 - 7x + 5$
Product = $[3x^4 + 2x^2 - 7x + 5]$
 = $3x^4 + 2x^2 - 7x + 5$

Number of polynomials -> 1
Input polynomial 1 -> $5 + 2x^2 + 3x^5 - x$
Sum = $[3x^5 + 2x^2 - x + 5]$
 = $3x^5 + 2x^2 - x + 5$
Product = $[3x^5 + 2x^2 - x + 5]$
 = $3x^5 + 2x^2 - x + 5$

Number of polynomials -> 1
Input polynomial 1 -> $3x^2 + 4x^2 - x + 6$
Sum = $[7x^2 - x + 6]$
 = $7x^2 - x + 6$
Product = $[7x^2 - x + 6]$
 = $7x^2 - x + 6$

Number of polynomials -> 1
Input polynomial 1 -> $x^2 - x^2$
Sum = $[0]$
 = 0
Product = $[0]$
 = 0

Number of polynomials -> 1
Input polynomial 1 -> $3x^2 - 4x + 7$
Sum = $[3x^2 - 4x + 7]$
 = $3x^2 - 4x + 7$
Product = $[3x^2 - 4x + 7]$
 = $3x^2 - 4x + 7$

```
Number of polynomials -> 1
Input polynomial 1 -> 2X^3 - X + 1
Sum = [2x^3 - x + 1]
      = 2x^3 - x + 1
Product = [2x^3 - x + 1]
          = 2x^3 - x + 1
```

```
-----
Number of polynomials -> 1
Input polynomial 1 -> 3x^-2 + 5
Sum = [5 + 3x^-2]
      = 5 + 3x^-2
Product = [5 + 3x^-2]
          = 5 + 3x^-2
```

```
-----
Number of polynomials -> 10
Input polynomial 1 -> x
Input polynomial 2 -> x
Input polynomial 3 -> x
Input polynomial 4 -> x
Input polynomial 5 -> x
Input polynomial 6 -> x
Input polynomial 7 -> x
Input polynomial 8 -> x
Input polynomial 9 -> x
Input polynomial 10 -> x
Sum = [x] + [x] + [x] + [x] + [x] + [x] + [x] + [x] + [x] + [x]
      = 10x
Product = [x] * [x] * [x] * [x] * [x] * [x] * [x] * [x] * [x] * [x]
          = x^10
```

```
-----
Number of polynomials -> -1
Invalid input. Please try again.
```

```
Number of polynomials -> 1
Input polynomial 1 -> x
Sum = [x]
      = x
Product = [x]
          = x
```

Number of polynomials -> -1
Invalid input. Please try again.

Number of polynomials -> 1
Input polynomial 1 -> x
Sum = [x]
 = x
Product = [x]
 = x

Number of polynomials -> 11
Invalid input. Please try again.

Number of polynomials -> 1
Input polynomial 1 -> x
Sum = [x]
 = x
Product = [x]
 = x

Number of polynomials -> abc
Invalid input. Please try again.

Number of polynomials -> 1
Input polynomial 1 -> x
Sum = [x]
 = x
Product = [x]
 = x

```
Number of polynomials -> 1
Input polynomial 1 ->
Invalid input. Please try again.
Input polynomial 1 -> x + 1
Sum = [x + 1]
      = x + 1
Product = [x + 1]
          = x + 1
```

```
-----
Number of polynomials -> 1
Input polynomial 1 -> 3x^
Invalid input. Please try again.
Input polynomial 1 -> 3x^2 + 4y
Invalid input. Please try again.
Input polynomial 1 -> 3x^2 4x + 1
Invalid input. Please try again.
Input polynomial 1 -> x
Sum = [x]
      = x
Product = [x]
          = x
```

```
-----
Number of polynomials -> 2
Input polynomial 1 -> 3x^2 + 1
Input polynomial 2 -> 4x - 5
Sum = [3x^2 + 1] + [4x - 5]
      = 3x^2 + 4x - 4
Product = [3x^2 + 1] * [4x - 5]
          = 12x^3 - 15x^2 + 4x - 5
```

```
-----
Number of polynomials -> 2
Input polynomial 1 -> 3x^2 + 2x + 1
Input polynomial 2 -> 5x^2 - 2x + 4
Sum = [3x^2 + 2x + 1] + [5x^2 - 2x + 4]
      = 8x^2 + 5
Product = [3x^2 + 2x + 1] * [5x^2 - 2x + 4]
          = 15x^4 + 4x^3 + 13x^2 + 6x + 4
```

```
-----  
Number of polynomials -> 2  
Input polynomial 1 ->  $2x^3 - 7$   
Input polynomial 2 ->  $-2x^3 + 7$   
Sum =  $[2x^3 - 7] + [-2x^3 + 7]$   
      = 0  
Product =  $[2x^3 - 7] * [-2x^3 + 7]$   
          =  $-4x^6 + 28x^3 - 49$   
-----
```

```
Number of polynomials -> 2  
Input polynomial 1 ->  $x + 1$   
Input polynomial 2 ->  $x + 1$   
Sum =  $[x + 1] + [x + 1]$   
      =  $2x + 2$   
Product =  $[x + 1] * [x + 1]$   
          =  $x^2 + 2x + 1$ 
```